

Teoria algorytmów i obliczeń - laboratorium

Zadanie laboratoryjne - multigrafy

Amadeusz Nowak

Andrii Voznesenskyi

Oleh Kiprik

Piotr Padamczyk

7 stycznia 2025

Spis treści

1	Definicje	2
2	Metody aproksymacyjne	3
2.1	Definicja dużego grafu	3
2.2	Kryteria formalne dla dużych grafów	4
2.3	Przykłady dużych grafów w implementacji	4
2.4	Zastosowanie metod aproksymacyjnych do dużych grafów	4
2.5	Wartości graniczne w implementacji	4
2.6	Znajdowanie cykli	4
2.7	Liczenie cykli Hamiltona	5
2.8	Obliczanie metryki między grafami	5
2.9	Rozszerzenie do cyklu Hamiltona	5
2.10	Maksymalny cykl	5
3	Przykłady działania programu	5
4	Opis algorytmów	9
4.1	Algorytm obliczania rozmiaru multigrafu	9
4.2	Algorytm wyszukiwania cykli w multigrafie	9
4.3	Algorytm liczenia cykli Hamiltona	10
4.4	Algorytm obliczania metryki pomiędzy dwoma multigrafami	10
4.5	Algorytm minimalnego rozszerzenia dla cyklu Hamiltona	11
5	Metody aproksymacyjne	11
5.1	Znajdowanie cykli w multigrafie	12
5.2	Liczenie cykli Hamiltona	12
5.3	Obliczanie metryki między dwoma multigrafami	13
5.4	Minimalne rozszerzenie do cyklu Hamiltona	13
5.5	Wykrywanie maksymalnego cyklu	14
5.6	Podsumowanie metod aproksymacyjnych	14
6	Dowód poprawności algorytmów	14
6.1	Dowód poprawności algorytmu obliczania rozmiaru multigrafu	14
6.2	Dowód poprawności algorytmu wyszukiwania cykli w multigrafie	15
6.3	Dowód poprawności algorytmu liczenia cykli Hamiltona	16
6.4	Dowód poprawności algorytmu obliczania metryki pomiędzy dwoma multigrafami	16
6.5	Dowód poprawności algorytmu minimalnego rozszerzenia dla cyklu Hamiltona	17

7 Dowód poprawności algorytmów aproksymacyjnych	18
7.1 Dowód poprawności algorytmu aproksymacyjnego znajdowania cykli w multigrafie . .	18
7.2 Dowód poprawności algorytmu aproksymacyjnego liczenia cykli Hamiltona	19
7.3 Dowód poprawności algorytmu aproksymacyjnego obliczania metryki między dwoma grafami	21
7.4 Dowód poprawności algorytmu aproksymacyjnego minimalnego rozszerzenia do cyklu Hamiltona	22
7.5 Dowód poprawności algorytmu aproksymacyjnego wykrywania maksymalnego cyklu .	23
8 Instrukcja obsługi	24
8.1 Struktura projektu	24
8.2 Kompilacja projektu	24
8.3 Uruchamianie programu głównego	24
8.4 Opis funkcji programu głównego	25
8.5 Uruchamianie generatora grafów	26
8.6 Przykłady użycia generatora grafów	26
8.7 Uruchamianie testów	26
9 Wnioski	26

1 Definicje

Definicja 1: Multigraf.

Multigraf to uporządkowana para $G = (V, E)$, gdzie:

- V jest skończonym zbiorem wierzchołków,
- E jest multizbiorem krawędzi, z których każda łączy parę wierzchołków z V . Krawędzie mogą się powtarzać, co oznacza możliwość występowania wielokrotnych krawędzi między tą samą parą wierzchołków. Nie rozważamy krawędzi w postaci pętli tj. krawędzie (v, v) .

Definicja 2: Rozmiar multigrafu.

Rozmiar multigrafu $G = (V, E)$, oznaczany $|E|$, to całkowita liczba krawędzi w multigrafie, uwzględniająca wszystkie wielokrotne krawędzie. Formalnie:

$$|E| = \sum_{(u,v) \in V \times V} \text{Liczność}((u, v), E),$$

gdzie $\text{Liczność}((u, v), E)$ to liczba wystąpień krawędzi (u, v) w E .

Definicja 3: Cykl w multigrafie.

Cykl w multigrafie $G = (V, E)$ to ciąg wierzchołków $v_1, v_2, \dots, v_k, v_1$, spełniający następujące warunki:

- $v_i \in V$ dla $i = 1, 2, \dots, k$,
- $(v_i, v_{i+1}) \in E$ dla $i = 1, 2, \dots, k - 1$, oraz $(v_k, v_1) \in E$,
- $v_i \neq v_j$ dla $1 \leq i < j \leq k$, z wyjątkiem $v_1 = v_k$.

Definicja 4: Maksymalny cykl w multigrafie.

Maksymalny cykl w multigrafie G to cykl, który zawiera największą liczbę wierzchołków spośród wszystkich cykli w G .

Definicja 5: Cykl Hamiltona.

Cykl Hamiltona w multigrafie $G = (V, E)$ to cykl, który odwiedza każdy wierzchołek z V dokładnie raz i powraca do wierzchołka początkowego. Formalnie, jest to ciąg $v_1, v_2, \dots, v_{|V|}, v_1$, taki że:

- $v_i \in V$ dla $i = 1, 2, \dots, |V|$,
- $(v_i, v_{i+1}) \in E$ dla $i = 1, 2, \dots, |V| - 1$, oraz $(v_{|V|}, v_1) \in E$,
- $v_i \neq v_j$ dla $1 \leq i < j \leq |V|$.

Definicja 6: Minimalne rozszerzenie multigrafu.

Minimalne rozszerzenie multigrafu $G = (V, E)$ to najmniejsza liczba krawędzi, które należy dodać do G , aby istniał w nim cykl Hamiltona.

Definicja 7: Metryka w przestrzeni multigrafów.

Metryką w przestrzeni multigrafów nazwiemy minimalną liczbą operacji potrzebnych do przekształcenia jednego grafu w drugi. Do operacji zaliczamy:

- dodanie krawędzi (również wielokrotnie),
- usunięcie krawędzi,
- dodanie wierzchołka,
- usunięcie wierzchołka,
- zamianę wierzchołków.

$$d(G, H) = \min_{\text{wszystkie sekwencje edycji}} \sum_{\text{operacje edycji grafu}} f_{\text{cost}}(\text{operacja}),$$

Przyjmujemy koszt wszystkich operacji edycji jako 1, oprócz zamiany wierzchołków, która ma koszt 0.

2 Metody aproksymacyjne

Rozważamy problem obliczeniowy na dużych multigrafach $G = (V, E)$, gdzie klasyczne algorytmy deterministyczne mają zbyt dużą złożoność obliczeniową. Z tego powodu stosuje się metody aproksymacyjne, które dają oszacowania wyników w ograniczonym czasie.

2.1 Definicja dużego grafu

W kontekście niniejszej implementacji, za *duży graf* uznaje się graf $G = (V, E)$, który spełnia poniższe kryteria:

- Liczba wierzchołków $|V|$ przekracza zdefiniowany próg THRESHOLD, tj. $|V| > \text{THRESHOLD}$, gdzie w implementacji $\text{THRESHOLD} = 10$.
- Liczba krawędzi $|E|$ jest proporcjonalna do $|V|^2$, co wskazuje na gęsty graf.

W szczególności, THRESHOLD jest parametrem definiowanym w kodzie źródłowym jako:

```
#define THRESHOLD 10
```

Oznacza to, że dla grafów o $|V| \leq \text{THRESHOLD}$ stosowane są algorytmy deterministyczne, natomiast dla $|V| > \text{THRESHOLD}$ stosowane są algorytmy aproksymacyjne.

2.2 Kryteria formalne dla dużych grafów

Graf G uznaje się za duży, jeśli spełnia przynajmniej jeden z poniższych warunków:

1. $|V| \gg \text{THRESHOLD}$, przy czym $|E| \approx O(|V|^2)$, co wskazuje na graf gęsty.
2. Rozmiar grafu w sensie pamięciowym przekracza możliwości operacyjne dostępnego środowiska obliczeniowego, tj.:

$$\text{Memory}(G) > \text{Available_Memory}.$$

3. Algorytmy deterministyczne na grafie G mają złożoność obliczeniową wyższą niż $O(|V|!)$, co czyni ich wykonanie niepraktycznym w rozsądnym czasie.

2.3 Przykłady dużych grafów w implementacji

W ramach implementacji, duże grafy mogą być reprezentowane jako:

- *Gęste grafy* (dense graphs), gdzie:

$$\frac{|E|}{|V|(|V| - 1)} \geq \alpha, \quad \alpha \in (0.1, 1].$$

- *Rzadkie grafy* (sparse graphs) o dużej liczbie wierzchołków $|V|$, dla których:

$$|E| \ll |V|^2, \quad |V| > \text{THRESHOLD}.$$

- *Multigrafy*, gdzie liczba wielokrotnych krawędzi jest znaczna, tj.:

$$\sum_{(u,v) \in V \times V} \text{Liczność}((u,v), E) \gg |V|.$$

2.4 Zastosowanie metod aproksymacyjnych do dużych grafów

Dla grafów spełniających powyższe kryteria, algorytmy deterministyczne są zastępowane przez metody aproksymacyjne w celu redukcji złożoności obliczeniowej. Przykłady takich algorytmów to:

- Losowe próbkowanie wierzchołków lub krawędzi w celu znalezienia cykli.
- Przybliżone obliczenia metryki odległości grafów za pomocą algorytmu symulowanego wyżarzania (Simulated Annealing).
- Heurystyczne dodawanie krawędzi w procesie rozszerzenia grafu do posiadania cyklu Hamiltona.

2.5 Wartości graniczne w implementacji

W implementacji za duże grafy uznaje się:

1. Grafy o liczbie wierzchołków $|V| > 10$.
2. Grafy, dla których algorytmy deterministyczne, takie jak pełne przeszukiwanie przestrzeni cykli, mają złożoność obliczeniową $O(|V|!)$.

Próg $\text{THRESHOLD} = 10$ może być zmieniany w zależności od dostępnych zasobów sprzętowych.

2.6 Znajdowanie cykli

Problem polega na znalezieniu wszystkich cykli w multigrafie. Formalnie:

$$\mathcal{C}(G) = \{C \subseteq V : \forall v_i, v_{i+1} \in C, (v_i, v_{i+1}) \in E \text{ oraz } v_{|C|} = v_1\}.$$

Metoda aproksymacyjna wybiera podzbiór $V' \subseteq V$ oraz analizuje jedynie podgraf $G' = (V', E')$, gdzie $E' = \{(u, v) \in E : u, v \in V'\}$. Wybór V' może być losowy lub deterministyczny, np. $|V'| = \lceil \alpha |V| \rceil$, gdzie $\alpha \in (0, 1]$ jest współczynnikiem redukcji.

2.7 Liczenie cykli Hamiltona

Problem polega na wyznaczeniu liczby cykli Hamiltona, czyli cykli zawierających wszystkie wierzchołki:

$$\mathcal{H}(G) = \{C \in \mathcal{C}(G) : |C| = |V|\}.$$

Aproksymacja tego problemu opiera się na zastosowaniu losowych prób T , w których generujemy losowe permutacje π zbioru V . Każda permutacja jest sprawdzana pod kątem spełnienia warunku cyklu:

$$C_\pi = (v_{\pi(1)}, v_{\pi(2)}, \dots, v_{\pi(|V|)}, v_{\pi(1)}) \in \mathcal{H}(G).$$

Złożoność zredukowana jest przez ograniczenie liczby prób $|T| \ll |V|!$.

2.8 Obliczanie metryki między grafami

W celu znalezienia metryki między dwoma grafami $G_1 = (V_1, E_1)$ i $G_2 = (V_2, E_2)$ generujemy wszystkie możliwe permutacje wierzchołków grafu o większej liczbie wierzchołków. Dla każdej permutacji π obliczamy metrykę $d(G_1, G_2)$ jako minimalną liczbę operacji edycji, które przekształcają graf G_1 w graf G_2 . Metryka jest minimalizowana poprzez wybór permutacji, która daje najmniejszą wartość metryki. Wiąże się to jednak z dużą złożonością obliczeniową, dlatego wykorzystywane są również metody aproksymacyjne.

Symulowane wyżarzanie to probabilistyczna technika optymalizacji, która pozwala aproksymować globalne optimum funkcji, dzięki czemu doskonale nadaje się do aproksymacji odległości grafów. Proces rozpoczyna się od inicjalizacji losowej permutacji wierzchołków grafu i obliczenia początkowej metryki `required_operations`. Na każdym kroku generowana jest sąsiednia permutacja poprzez zamianę dwóch wierzchołków. Jeśli nowa permutacja daje niższą wartość `required_operations`, jest akceptowana. W przeciwnym razie może być zaakceptowana z pewnym prawdopodobieństwem, które maleje wraz z upływem czasu, co pozwala algorytmowi unikać lokalnych minimów. Harmonogram chłodzenia stopniowo zmniejsza prawdopodobieństwo akceptacji gorszych rozwiązań, zapewniając zbieżność. Algorytm kończy działanie po ustalonej liczbie iteracji lub gdy temperatura osiągnie określony niski próg.

2.9 Rozszerzenie do cyklu Hamiltona

Minimalne rozszerzenie grafu $G = (V, E)$ polega na dodaniu minimalnej liczby krawędzi E^* , tak aby graf $G' = (V, E \cup E^*)$ spełniał:

$$\mathcal{H}(G') \neq \emptyset.$$

Aproksymacja opiera się na heurystycznym dodawaniu krawędzi $(u, v) \in \{V \times V \setminus E\}$, gdzie u, v mają minimalne stopnie w G .

2.10 Maksymalny cykl

Problem maksymalnego cyklu polega na znalezieniu cyklu $C_{\max}$ o największej liczbie wierzchołków:

$$C_{\max} \in \arg \max_{C \in \mathcal{C}(G)} |C|.$$

Algorytm aproksymacyjny korzysta z losowego przeszukiwania grafu, ograniczając głębokość rekursji d oraz wykluczając powtarzające się ścieżki. Wartość d jest parametrem kontrolującym dokładność metody.

3 Przykłady działania programu

Przykład 1: Multigraf bez pętli i z cyklem Hamiltona

Argumenty wejściowe:

- Liczba grafów: 1
- Liczba wierzchołków w grafie: 3
- Macierz sąsiedztwa:

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 2 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

gdzie $A[i][j]$ oznacza liczbę krawędzi pomiędzy wierzchołkiem i a j .

Wynik:

- Rozmiar grafu (liczba krawędzi): 8
- Wszystkie cykle:
 $0 \rightarrow 1 \rightarrow 0$
 $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$
 $0 \rightarrow 2 \rightarrow 1 \rightarrow 0$
- Cykle Hamiltona:
 $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$
 $0 \rightarrow 2 \rightarrow 1 \rightarrow 0$
- Minimalne rozszerzenie dla cyklu Hamiltona: 0
- Maksymalna długość cyklu: 3

Przykład 2: Multigraf z pętlami i cyklami różnej długości

Argumenty wejściowe:

- Liczba grafów: 2
- Graf 1: 3 wierzchołki, macierz sąsiedztwa:

$$A_1 = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

- Graf 2: 4 wierzchołki, macierz sąsiedztwa:

$$A_2 = \begin{bmatrix} 0 & 2 & 0 & 1 \\ 2 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

Wynik:

- **Graf 1:**
 - Rozmiar grafu: 4
 - Wszystkie cykle: Brak
 - Cykl Hamiltona: Brak
 - Minimalne rozszerzenie dla cyklu Hamiltona: 1
 - Maksymalna długość cyklu: 0
- **Graf 2:**

- Rozmiar grafu: 10
 - Wszystkie cykle:
 - $0 \rightarrow 1 \rightarrow 0$
 - $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$
 - $0 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$
 - Cykl Hamiltona:
 - $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$
 - $0 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$
 - Minimalne rozszerzenie dla cyklu Hamiltona: 0
 - Maksymalna długość cyklu: 4
- Metryka podobieństwa pomiędzy grafem 1 i grafem 2: 1.167

Przykład 3: Minimalne rozszerzenie dla cyklu Hamiltona

Argumenty wejściowe:

- Liczba grafów: 3
- Graf 1: 3 wierzchołki, macierz sąsiedztwa:

$$A_1 = \begin{bmatrix} 0 & 2 & 0 \\ 2 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix}$$

- Graf 2: 4 wierzchołki, macierz sąsiedztwa:

$$A_2 = \begin{bmatrix} 0 & 3 & 1 & 0 \\ 3 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 2 & 1 & 0 \end{bmatrix}$$

- Graf 3: 4 wierzchołki, macierz sąsiedztwa:

$$A_3 = \begin{bmatrix} 0 & 2 & 0 & 0 \\ 2 & 0 & 1 & 2 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Wynik:

- **Graf 1:**
 - Rozmiar grafu: 6
 - Wszystkie cykle: $0 \rightarrow 1 \rightarrow 0$
 - Cykl Hamiltona: Brak
 - Minimalne rozszerzenie dla cyklu Hamiltona: 1
 - Maksymalna długość cyklu: 2
- **Graf 2:**
 - Rozmiar grafu: 12
 - Wszystkie cykle:
 - $0 \rightarrow 1 \rightarrow 0$
 - $0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0$

- Cykl Hamiltona:
 $0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0$
- Minimalne rozszerzenie dla cyklu Hamiltona: 0
- Maksymalna długość cyklu: 4

- **Graf 3:**

- Rozmiar grafu: 10
- Wszystkie cykle: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$
- Cykl Hamiltona: Nie
- Minimalne rozszerzenie dla cyklu Hamiltona: 1
- Maksymalna długość cyklu: 3

- Metryka podobieństwa pomiędzy grafem 1 i grafem 2: 0.833
- Metryka podobieństwa pomiędzy grafem 2 i grafem 3: 0.500

Przykład 4: Metryka w przestrzeni multigrafów

Argumenty wejściowe:

- Liczba grafów: 2
- Graf 1: 4 wierzchołki, macierz sąsiedztwa:

$$A_1 = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

- Graf 2: 3 wierzchołki, macierz sąsiedztwa:

$$A_2 = \begin{bmatrix} 0 & 2 & 1 \\ 2 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

Wynik:

- **Graf 1:**

- Rozmiar grafu: 4
- Wszystkie cykle: $0 \rightarrow 1 \rightarrow 3 \rightarrow 0$
- Cykl Hamiltona: Nie
- Minimalne rozszerzenie dla cyklu Hamiltona: 1
- Maksymalna długość cyklu: 3

- **Graf 2:**

- Rozmiar grafu: 4
- Wszystkie cykle: $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$
- Cykl Hamiltona: Tak $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$
- Minimalne rozszerzenie dla cyklu Hamiltona: 0
- Maksymalna długość cyklu: 3

- Metryka podobieństwa pomiędzy grafem 1 i grafem 2: 5
- Metryka podobieństwa pomiędzy grafem 1 i grafem 2 liczona metodą aproksymacyjną: 5

4 Opis algorytmów

4.1 Algorytm obliczania rozmiaru multigrafu

Algorithm 1 Obliczanie rozmiaru multigrafu

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa A

Ensure: Rozmiar multigrafu $|E|$

```
1:  $|E| \leftarrow 0$ 
2: for each wierzchołki  $i, j$  w  $V$ 
3:    $|E| \leftarrow |E| + A[i][j]$ 
4: end for
5: return  $|E|$ 
```

Złożoność obliczeniowa algorytmu wynosi $O(n^2)$, gdzie n to liczba wierzchołków w multigrafie. Wynika to z iteracji po całej macierzy sąsiedztwa A o wymiarach $n \times n$.

4.2 Algorytm wyszukiwania cykli w multigrafie

Algorithm 2 Znajdowanie cykli w multigrafie

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa A

Ensure: Liczba cykli w G

```
1: cycle_count  $\leftarrow 0$ 
2: Utwórz pusty zbiór visited
3: function DFS(( u, start, path ))
4:   if length(path) > 2 and  $u = \text{start}$  then
5:     cycle_count  $\leftarrow$  cycle_count + 1
6:     return
7:   end if
8:   for each  $v$  sąsiadujący z  $u$ 
9:     if  $v \notin \text{visited}$  then
10:      Dodaj  $v$  do visited
11:      DFS(( v, start, path + [v] ))
12:      Usuń  $v$  z visited
13:     end if
14:   end for
15: end function
16: for each wierzchołek  $u \in V$ 
17:   visited  $\leftarrow \{u\}$ 
18:   DFS(( u, u, [u] ))
19: end for
20: return cycle_count
```

Złożoność obliczeniowa algorytmu DFS wynosi $O(V + E)$. W przypadku multigrafu, gdzie V może być większe niż E , złożoność obliczeniowa wynosi $O(V^2)$. Ponieważ algorytm DFS jest wywoływany dla każdego wierzchołka, złożoność obliczeniowa całego algorytmu wynosi $O(V(V + E))$ lub $O(V^3)$ dla przytoczonego drugiego przypadku.

4.3 Algorytm liczenia cykli Hamiltona

Algorithm 3 Liczenie cykli Hamiltona w multigrafie

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa A

Ensure: Liczba cykli Hamiltona

```
1: count  $\leftarrow 0$ 
2: function HAMILTONIANBACKTRACK(( u, visited, depth ))
3:   if depth =  $|V|$  and  $A[u][start] > 0$  then
4:     count  $\leftarrow$  count + 1
5:     return
6:   end if
7:   for each  $v$  sąsiadujący z  $u$ 
8:     if  $v \notin$  visited then
9:       Dodaj  $v$  do visited
10:      HAMILTONIANBACKTRACK(( v, visited, depth + 1 ))
11:      Usuń  $v$  z visited
12:    end if
13:  end for
14: end function
15: for each wierzchołek  $u \in V$ 
16:   visited  $\leftarrow \{u\}$ 
17:   HAMILTONIANBACKTRACK(( u, visited, 1 ))
18: end for
19: return count
```

Złożoność obliczeniowa algorytmu liczenia cykli Hamiltona wynosi $O(V!)$. Wynika to z wykonywania funkcji rekurencyjnej do znajdowania wszystkich ścieżek. Z każdym poziomem głębokości funkcji rekurencyjnej mamy o jeden mniej wierzchołek do wyboru, co daje $V!$ możliwości.

4.4 Algorytm obliczania metryki pomiędzy dwoma multigrafami

Algorithm 4 Obliczanie metryki pomiędzy dwoma multigrafami

Require: $G = (V_G, E_G)$ oraz $H = (V_H, E_H)$ - dwa multigrafy reprezentowane przez macierze sąsiedztwa A_G i A_H oraz $|V_G| \geq |V_H|$.

Ensure: Metryka pomiędzy G i H

```
1: metric  $\leftarrow \infty$ 
2: for each permutacja  $\pi$  wierzchołków  $V_G$ 
3:   tmp  $\leftarrow 0$ 
4:   for each wierzchołki  $i, j$  w  $V_H$ 
5:     if  $i \geq V_H$  OR  $j \geq V_H$  then
6:       tmp  $\leftarrow$  tmp +  $|A_G[\pi[i]][\pi[j]]|$ 
7:     else if then
8:       tmp  $\leftarrow$  tmp +  $|A_G[\pi[i]][\pi[j]] - A_H[i][j]|$ 
9:     end if
10:  end for
11: end for
12: metric  $\leftarrow$  metric +  $|V_G| - |V_H|$ 
13: return metric
```

Złożoność algorytmu obliczania metryki dla grafów $G = (V_G, E_G)$ oraz $H = (V_H, E_H)$ wynosi $O(|V_G|!)$, gdzie graf G ma większą liczbę wierzchołków niż H .

4.5 Algorytm minimalnego rozszerzenia dla cyklu Hamiltona

Algorithm 5 Minimalne rozszerzenie dla cyklu Hamiltona

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa A

Ensure: Minimalna liczba krawędzi do dodania, aby uzyskać cykl Hamiltona

```
1: min_edges  $\leftarrow \infty$ 
2: function HASHAMILTONIANCYCLE( $G, V$ )
3:   cycle_count  $\leftarrow$  COUNTHAMILTONIANCYCLES( $G, V$ )
4:   return cycle_count > 0
5: end function
6: function EXTENDGRAPH( $G, \text{added\_edges}$ )
7:   if added_edges  $\geq$  min_edges then
8:     return
9:   end if
10:  if HASHAMILTONIANCYCLE( $G, V$ ) then
11:    min_edges  $\leftarrow$  added_edges
12:    return
13:  end if
14:  for each  $u, v \in V$ 
15:    if  $u \neq v$  i  $A[u][v] = 0$  then
16:      Dodaj krawędź  $(u, v)$  do  $A$ 
17:      EXTENDGRAPH( $G, \text{added\_edges} + 1$ )
18:      Usuń krawędź  $(u, v)$  z  $A$ 
19:    end if
20:  end for
21: end function
22: EXTENDGRAPH( $G, 0$ )
23: return min_edges
```

Złożoność algorytmu wynosi $O(2^m \cdot n!)$, gdzie n to liczba wierzchołków, a $m = \binom{n}{2} - |E|$. Wynika to z tego, że tworzymy grafy poprzez dodanie jeszcze nie istniejących krawędzi, i dla każdego takiego grafu wykonujemy algorytm 3.

5 Metody aproksymacyjne

Aproksymacyjne metody pozwalają na zmniejszenie złożoności obliczeniowej algorytmów, dzięki czemu możliwe jest szybkie przetwarzanie dużych multigrafów $G = (V, E)$. Poniżej opisano szczegółowo podejścia aproksymacyjne dla problemów, które zostały zaimplementowane w kodzie.

5.1 Znajdowanie cykli w multigrafie

Algorithm 6 Aproksymacyjne znajdowanie cykli w multigrafie

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa A , T - liczba prób

Ensure: Przybliżona liczba cykli w G

```
1: cycle_count  $\leftarrow$  0
2: for  $t = 1$  do  $T$  do
3:   Wybierz losowy wierzchołek  $u \in V$ 
4:   for each  $v \in V$  sąsiadujący z  $u$ 
5:     if  $(u, v) \in E$  then
6:       Dodaj  $u, v, u$  jako cykl
7:       cycle_count  $\leftarrow$  cycle_count + 1
8:     end if
9:   end for
10: end for
11: return cycle_count
```

Złożoność obliczeniowa wynosi $O(T \cdot \deg(V))$, gdzie T to liczba prób, a $\deg(V)$ to średni stopień wierzchołka.

5.2 Liczenie cykli Hamiltona

Algorithm 7 Aproksymacyjne liczenie cykli Hamiltona

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa A , T - liczba prób

Ensure: Przybliżona liczba cykli Hamiltona

```
1: hamiltonian_count  $\leftarrow$  0
2: for  $t = 1$  do  $T$  do
3:   Wygeneruj losową permutację  $\pi$  zbioru  $V$ 
4:   Sprawdź, czy  $\pi$  tworzy cykl Hamiltona:
5:   for  $i = 1$  do  $|V| - 1$  do
6:     if  $(v_{\pi(i)}, v_{\pi(i+1)}) \notin E$  then
7:       Przerwij pętlę
8:     end if
9:   end for
10:  if  $(v_{\pi(|V|)}, v_{\pi(1)}) \in E$  then
11:    hamiltonian_count  $\leftarrow$  hamiltonian_count + 1
12:  end if
13: end for
14: return hamiltonian_count
```

Złożoność obliczeniowa wynosi $O(T \cdot |V|)$, gdzie T to liczba prób.

5.3 Obliczanie metryki między dwoma multigrafami

Algorithm 8 Aproksymacyjne obliczanie metryki między dwoma grafami

Require: $G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$ - dwa multigrafy reprezentowane przez macierze sąsiedztwa, gdzie $|V_1| \geq |V_2|$, T - liczba prób, *InitialTemperature* - początkowa temperatura, *CoolingRate* - współczynnik chłodzenia (w implementacji *CoolingRate* = 0.99)

Ensure: Przybliżona wartość metryki między G_1 a G_2

```
1: Wygeneruj losową permutację  $\pi$  zbioru  $V_1$ 
2: Oblicz początkową metrykę metric dla  $\pi$ 
3:  $\text{best\_metric} \leftarrow \text{metric}$ 
4:  $\text{temperature} \leftarrow \text{InitialTemperature}$ 
5: for  $t = 1$  do  $T$  do
6:   Zamień losowe wierzchołki  $u, v \in V_1$  w permutacji  $\pi$ 
7:   Oblicz metrykę  $\text{metric}'$  dla nowego  $\pi$ 
8:   if  $\text{metric}' < \text{metric}$  OR  $\text{rand}(0, 1) < \exp(\frac{\text{metric} - \text{metric}'}{t})$  then
9:      $\text{metric} \leftarrow \text{metric}'$ 
10:    if  $\text{metric} < \text{best\_metric}$  then
11:       $\text{best\_metric} \leftarrow \text{metric}$ 
12:    else if then
13:      Przywróć poprzednią permutację
14:    end if
15:  end if
16:   $\text{temperature} \leftarrow \text{temperature} \times \text{CoolingRate}$ 
17: end for
18: return  $\text{best\_metric}$ 
```

Złożoność obliczeniowa wynosi $O(T \cdot n)$, gdzie $n = \max(|V_1|, |V_2|)$. Wykonujemy T iteracji, a w każdej z nich zamieniamy wierzchołki w permutacji, co wiąże się z liniową złożonością obliczeniową.

5.4 Minimalne rozszerzenie do cyklu Hamiltona

Algorithm 9 Aproksymacyjne minimalne rozszerzenie do cyklu Hamiltona

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa

Ensure: Minimalna liczba krawędzi do dodania

```
1:  $\text{added\_edges} \leftarrow 0$ 
2: for  $u = 1$  do  $|V|$  do
3:   for  $v = u + 1$  do  $|V|$  do
4:     if  $A[u][v] = 0$  then
5:       Dodaj  $(u, v)$  do  $E$ 
6:        $\text{added\_edges} \leftarrow \text{added\_edges} + 1$ 
7:       Przerwij wewnętrzną pętlę
8:     end if
9:   end for
10: end for
11: return  $\text{added\_edges}$ 
```

Złożoność obliczeniowa wynosi $O(|V|^2)$.

5.5 Wykrywanie maksymalnego cyklu

Algorithm 10 Aproksymacyjne wykrywanie maksymalnego cyklu

Require: $G = (V, E)$ - multigraf reprezentowany przez macierz sąsiedztwa, T - liczba prób

Ensure: Maksymalny cykl w G

```
1: max_cycle_length  $\leftarrow$  0
2: for  $t = 1$  do  $T$  do
3:   Wybierz losowy wierzchołek  $u$ 
4:   Wykonaj DFS do maksymalnej głębokości  $d$ 
5:   Zapisz długość cyklu, jeśli  $u$  zostanie osiągnięte ponownie
6:   if cycle_length  $>$  max_cycle_length then
7:     Zaktualizuj max_cycle_length
8:   end if
9: end for
10: return max_cycle_length
```

Złożoność obliczeniowa wynosi $O(T \cdot |V|)$, gdzie T to liczba prób.

5.6 Podsumowanie metod aproksymacyjnych

Metody aproksymacyjne opisane w tej sekcji pozwalają na szybsze rozwiązanie problemów na dużych grafach. Dzięki ograniczeniu liczby prób T , losowemu próbkowaniu oraz uproszczeniu obliczeń, możliwe jest znalezienie przybliżonych wyników w czasie wykładniczo mniejszym niż w przypadku klasycznych algorytmów deterministycznych.

6 Dowód poprawności algorytmów

6.1 Dowód poprawności algorytmu obliczania rozmiaru multigrafu

Aby formalnie udowodnić poprawność algorytmu OBLICZROZMIARMULTIGRAFU (Algorytm 1), wykorzystamy indukcję matematyczną.

Teza: Dla dowolnej macierzy sąsiedztwa A grafu $G = (V, E)$, algorytm poprawnie oblicza liczbę krawędzi $|E| = \sum_{i=1}^n \sum_{j=1}^n A[i][j]$, gdzie $n = |V|$.

Podstawa indukcji: Rozważmy graf o $n = 1$ wierzchołku. Macierz sąsiedztwa A ma wymiar 1×1 i zawiera jedną wartość $A[1][1]$, która reprezentuje liczbę pętli w grafie.

Dla $n = 1$, algorytm: 1. Inicjalizuje $|E| \leftarrow 0$. 2. Iteruje po macierzy A , która zawiera jedynie $A[1][1]$. 3. Dodaje $A[1][1]$ do $|E|$.

Po zakończeniu iteracji algorytm zwraca $|E| = A[1][1]$, co jest zgodne z definicją liczby krawędzi grafu. Podstawa indukcji została zatem wykazana.

Krok indukcyjny: Załóżmy, że algorytm poprawnie oblicza liczbę krawędzi dla dowolnego grafu o $n = k$ wierzchołkach. Udowodnijmy, że algorytm działa poprawnie dla $n = k + 1$ wierzchołków.

Założenie indukcyjne: Dla grafu $G_k = (V_k, E_k)$ o k wierzchołkach macierz sąsiedztwa A_k jest rozmiaru $k \times k$, a algorytm zwraca poprawną wartość:

$$|E_k| = \sum_{i=1}^k \sum_{j=1}^k A_k[i][j].$$

Dowód dla $n = k + 1$: Rozważmy graf $G_{k+1} = (V_{k+1}, E_{k+1})$ z macierzą sąsiedztwa A_{k+1} , która jest rozmiaru $(k + 1) \times (k + 1)$. Macierz A_{k+1} można przedstawić jako:

$$A_{k+1} = \begin{bmatrix} A_k & b \\ c^\top & d \end{bmatrix},$$

gdzie: - A_k jest podmacierzą $k \times k$ dla grafu G_k , - b i $c^\top$ to wektory krawędzi pomiędzy nowym wierzchołkiem a istniejącymi k wierzchołkami, - d to liczba pętli w nowym wierzchołku.

Algorytm iteruje po wszystkich $(k+1)^2$ elementach macierzy A_{k+1} . W trakcie iteracji: 1. Sumuje wszystkie elementy $A_k[i][j]$ z podmacierzy A_k , zgodnie z założeniem indukcyjnym. 2. Dodaje wartości z wektorów b , $c^\top$, oraz d .

Łączna suma, jaką zwraca algorytm, wynosi:

$$|E_{k+1}| = \sum_{i=1}^k \sum_{j=1}^k A_k[i][j] + \sum_{i=1}^k b[i] + \sum_{j=1}^k c[j] + d.$$

Jest to zgodne z definicją liczby krawędzi w grafie o $k+1$ wierzchołkach:

$$|E_{k+1}| = \sum_{i=1}^{k+1} \sum_{j=1}^{k+1} A_{k+1}[i][j].$$

Wniosek: Na mocy zasady indukcji matematycznej algorytm OBLICZROZMIARMULTIGRAFU działa poprawnie dla dowolnej macierzy sąsiedztwa A grafu G .

6.2 Dowód poprawności algorytmu wyszukiwania cykli w multigrafie

Aby udowodnić poprawność algorytmu ZNAJDŹCYKLE (Algorytm 2), wykorzystamy indukcję matematyczną w oparciu o liczbę wierzchołków $n = |V|$.

Teza: Dla grafu $G = (V, E)$ reprezentowanego przez macierz sąsiedztwa A , algorytm ZNAJDŹCYKLE poprawnie identyfikuje wszystkie cykle, przy czym cykle są definiowane jako zamknięte ścieżki w grafie, w których każdy wierzchołek poza punktem początkowym jest odwiedzany najwyżej raz.

Podstawa indukcji: Rozważmy graf o $n = 1$ wierzchołku. Macierz sąsiedztwa A ma wymiar 1×1 , a jedyną potencjalną krawędzią jest pętla $A[1][1]$, która tworzy cykl o długości 1.

Algorytm: 1. Inicjalizuje $\text{cycle_count} \leftarrow 0$. 2. Rozpoczyna przeszukiwanie w głąb (DFS) od wierzchołka $u = 1$. 3. Sprawdza, czy długość aktualnej ścieżki path wynosi co najmniej 3 (co jest niemożliwe dla $n = 1$). 4. Kończy działanie, zwracając $\text{cycle_count} = 0$, co jest poprawne.

Podstawa indukcji została wykazana, ponieważ graf o jednym wierzchołku nie zawiera cykli o długości większej niż 2.

Krok indukcyjny: Załóżmy, że algorytm poprawnie identyfikuje wszystkie cykle w dowolnym grafie $G_k = (V_k, E_k)$ o $n = k$ wierzchołkach. Udowodnijmy, że działa poprawnie dla grafu $G_{k+1} = (V_{k+1}, E_{k+1})$ o $n = k+1$ wierzchołkach.

Założenie indukcyjne: Dla dowolnego grafu G_k , algorytm identyfikuje wszystkie cykle $C \subseteq G_k$, przy czym każdy cykl spełnia warunek:

$$C = \{v_1, v_2, \dots, v_m\}, \quad v_1 = v_m, \quad v_i \neq v_j \text{ dla } i \neq j.$$

Dowód dla $n = k+1$: Rozważmy graf $G_{k+1} = (V_{k+1}, E_{k+1})$ o $k+1$ wierzchołkach. Algorytm rozpoczyna przeszukiwanie w głąb (DFS) od każdego wierzchołka u i wykonuje następujące kroki: 1. Dodaje u do aktualnej ścieżki path i oznacza jako odwiedzony w zbiorze visited . 2. Rekurencyjnie eksploruje wszystkich sąsiadów v , którzy nie należą do visited . 3. Jeżeli wierzchołek startowy u zostanie osiągnięty po co najmniej dwóch krokach ($\text{length}(\text{path}) > 2$), algorytm zlicza cykl.

Każdy potencjalny cykl w G_{k+1} należy do jednej z dwóch kategorii: - $C \subseteq G_k$: Cykl istnieje wyłącznie w podgrafie G_k , a poprawność jego detekcji wynika z założenia indukcyjnego. - $C \not\subseteq G_k$: Cykl obejmuje nowy wierzchołek v_{k+1} .

Dla $C \not\subseteq G_k$, algorytm identyfikuje cykl C w sposób następujący: - Przeszukiwanie w głąb obejmuje wszystkie sąsiadujące wierzchołki $v \in V_k$, które mają krawędź do v_{k+1} (czyli $A[v][v_{k+1}] > 0$). - Warunek $u = \text{start}$ po powrocie do wierzchołka początkowego u gwarantuje, że cykl został zamknięty.

Złożoność rekurencji wynosi $O(k+1 + E_{k+1})$ dla każdego wierzchołka u , co zapewnia, że wszystkie krawędzie są odwiedzane dokładnie raz.

Wniosek: Na mocy zasady indukcji matematycznej algorytm ZNAJDŹCYKLE poprawnie identyfikuje wszystkie cykle w grafie o dowolnej liczbie wierzchołków n .

6.3 Dowód poprawności algorytmu liczenia cykli Hamiltona

Udowodnijmy poprawność algorytmu POLICZCYKLEHAMILTONA (Algorytm 3), który wykorzystuje technikę rekurencyjnego backtrackingu, aby znaleźć wszystkie cykle Hamiltona w grafie $G = (V, E)$.

Teza: Algorytm POLICZCYKLEHAMILTONA poprawnie identyfikuje wszystkie cykle Hamiltona w grafie G , czyli wszystkie permutacje wierzchołków $\pi \in V$ spełniające warunek:

$$\forall i (1 \leq i < n) : (v_i, v_{i+1}) \in E \text{ oraz } (v_n, v_1) \in E.$$

Podstawa indukcji: Rozważmy graf G_1 o $n = 1$ wierzchołku. Macierz sąsiedztwa A jest macierzą 1×1 , a jedyną potencjalną krawędzią jest pętla $A[1][1]$. Algorytm: 1. Inicjalizuje count $\leftarrow 0$. 2. Rozpoczyna przeszukiwanie w głąb (DFS) od $u = 1$. 3. Osiąga głębokość $\text{depth} = |V| = 1$, sprawdza $A[1][1]$ i jeśli $A[1][1] > 0$, zlicza cykl Hamiltona.

Dla G_1 , istnieje dokładnie jeden cykl Hamiltona (pętla wierzchołka na sobie), jeśli $A[1][1] > 0$. Algorytm zwraca poprawny wynik, count = 1, lub count = 0 w przypadku $A[1][1] = 0$.

Krok indukcyjny: Załóżmy, że algorytm działa poprawnie dla grafu $G_k = (V_k, E_k)$ o $n = k$ wierzchołkach. Udowodnijmy, że działa poprawnie dla $G_{k+1} = (V_{k+1}, E_{k+1})$.

Założenie indukcyjne: Algorytm identyfikuje wszystkie cykle Hamiltona w G_k , tj. każdą permutację π wierzchołków w V_k , spełniającą warunek:

$$\forall i (1 \leq i < k) : (v_i, v_{i+1}) \in E_k \text{ oraz } (v_k, v_1) \in E_k.$$

Dowód dla G_{k+1} : Rozważmy graf G_{k+1} , gdzie dodano wierzchołek v_{k+1} oraz potencjalne krawędzie (v_{k+1}, v_i) dla $v_i \in V_k$. Algorytm: 1. Rozpoczyna DFS od każdego $u \in V_{k+1}$. 2. Dodaje u do zbioru visited i ścieżki path. 3. Rekurencyjnie eksploruje sąsiadów v , które nie znajdują się w visited. 4. W głębokości $\text{depth} = |V| = k + 1$ sprawdza, czy istnieje krawędź powrotna $A[v_{k+1}][v_1]$.

Analiza poprawności: 1. Dla każdego $v \in V_{k+1}$, rekurencja odwiedza dokładnie $k+1$ wierzchołków w grafie. 2. Warunek zakończenia rekurencji ($\text{depth} = |V|$ i $A[u][\text{start}] > 0$) gwarantuje, że algorytm rozpoznaje tylko cykle Hamiltona. 3. Wszystkie potencjalne permutacje odwiedzenia wierzchołków π są eksplorowane dzięki iteracji $\forall v \in V_{k+1}$ oraz śledzeniu odwiedzonych wierzchołków w visited.

Redukcja do przestrzeni języków formalnych: Cykl Hamiltona można opisać w języku formalnym L_H :

$$L_H = \{\pi \in \Sigma^n : \forall i (1 \leq i < n) (v_i, v_{i+1}) \in E \text{ oraz } (v_n, v_1) \in E\}.$$

Algorytm można zredukować do maszyny Turinga M_H , która generuje wszystkie permutacje $\pi \in \Sigma^n$ i akceptuje je, jeśli $\pi \in L_H$. Maszyna M_H działa w czasie $O(n!)$ na wejściu długości n .

Złożoność: Dla każdego podzbioru odwiedzonych wierzchołków $S \subseteq V$, DFS wykonuje $O(n)$ kroków w celu eksploracji sąsiadów i $O(n^2)$ operacji w sprawdzaniu warunku zakończenia rekurencji. Łączna liczba podzbiorów wynosi 2^n , co prowadzi do złożoności $O(2^n \cdot n^2)$.

Wniosek: Na mocy indukcji matematycznej oraz zgodności z językiem L_H , algorytm POLICZCYKLEHAMILTONA poprawnie identyfikuje wszystkie cykle Hamiltona w grafie G .

6.4 Dowód poprawności algorytmu obliczania metryki pomiędzy dwoma multigrafami

Algorytm obliczania metryki pomiędzy dwoma multigrafami (OBLICZMETRYKĘ, Algorytm 5) liczy ilość potrzebnych operacji do przekształcenia jednego grafu w drugi.

Opis algorytmu Algorytm przebiega następująco:

1. Generujemy wszystkie permutacje π zbioru wierzchołków V_1 w porządku leksykograficznym.
2. Dla każdej permutacji π , obliczamy macierz sąsiedztwa $A(G_1^\pi)$ grafu permutowanego G_1^π .
3. Porównujemy macierz sąsiedztwa $A(G_1^\pi)$ z macierzą sąsiedztwa $A(G_2)$ grafu G_2 , aby określić liczbę wstawień i usunięć krawędzi i wierzchołków.
4. Wybieramy permutację π , która minimalizuje liczbę wstawień i usunięć niezbędnych do przekształcenia $A(G_1^\pi)$ w $A(G_2)$.

Dowód poprawności Niech $A(G)$ oznacza macierz sąsiedztwa grafu G . Odległość między dwoma grafami G_1 i G_2 definiuje się jako minimalną liczbę wstawień i usunięć wierzchołków i krawędzi, które są wymagane, aby przekształcić G_1 w G_2 .

Krok 1: Permutacja wierzchołków grafu G_1 . Generujemy wszystkie permutacje π zbioru wierzchołków V_1 , ponieważ może istnieć izomorfizm między grafami G_1 i G_2 , w którym wierzchołki grafu G_1 mają inne etykiety. Każda permutacja π daje inne rozmieszczenie wierzchołków, a tym samym inną macierz sąsiedztwa $A(G_1^\pi)$.

Krok 2: Obliczanie macierzy sąsiedztwa dla każdej permutacji. Dla każdej permutacji π obliczamy macierz sąsiedztwa $A(G_1^\pi)$ odpowiadającą grafowi G_1^π , który ma te same krawędzie co G_1 , ale wierzchołki są ponumerowane zgodnie z π .

Krok 3: Porównanie macierzy sąsiedztwa. Obliczamy liczbę wstawień i usunięć krawędzi i wierzchołków, które są wymagane, aby przekształcić $A(G_1^\pi)$ w $A(G_2)$. Wymaga to obliczenia sumy wartości bezwzględnych różnic między odpowiadającymi elementami macierzy sąsiedztwa $A(G_1^\pi)$ i $A(G_2)$.

Krok 4: Minimalizacja liczby operacji. Algorytm wybiera permutację π , która minimalizuje liczbę wstawień i usunięć wymaganych do dopasowania $A(G_1^\pi)$ do $A(G_2)$.

Poprawność algorytmu. Rozważając wszystkie permutacje wierzchołków grafu G_1 , algorytm gwarantuje, że zostanie znaleziona najlepsza możliwa zgodność między wierzchołkami G_1 i G_2 . Ponieważ porównanie macierzy sąsiedztwa dla każdej permutacji liczy dokładną liczbę wymaganych wstawień i usunięć, a algorytm wybiera permutację, która minimalizuje liczbę operacji, algorytm oblicza optymalne rozwiązanie problemu transformacji grafu.

W związku z tym algorytm jest poprawny i oblicza minimalną liczbę wstawień i usunięć krawędzi oraz wierzchołków, które są wymagane, aby przekształcić G_1 w G_2 .

6.5 Dowód poprawności algorytmu minimalnego rozszerzenia dla cyklu Hamiltona

Algorytm minimalnego rozszerzenia dla cyklu Hamiltona (MINIMALNEROZSZERZENIE, Algorytm 5) znajduje minimalną liczbę krawędzi, które należy dodać do grafu G , aby uzyskać graf zawierający cykl Hamiltona. Dowód poprawności algorytmu opiera się na kilku elementach teorii grafów, prze-strzeni języków formalnych oraz maszyn Turinga. Zastosujemy także indukcję matematyczną i analizę przestrzeni poszukiwań.

Teza: Algorytm MINIMALNEROZSZERZENIE poprawnie oblicza minimalną liczbę krawędzi k , które należy dodać do grafu $G = (V, E)$, aby uzyskać cykl Hamiltona.

Dowód: 1. Konstrukcja algorytmu:

Algorytm działa w sposób rekurencyjny. Dla każdego grafu G' , uzyskanego przez dodanie k krawędzi do G , algorytm: 1. Sprawdza, czy G' zawiera cykl Hamiltona, korzystając z funkcji pomocniczej HASHAMILTONIANCYCLE. 2. Jeśli G' zawiera cykl Hamiltona, zapisuje k jako kandydat na minimalną wartość i kończy dalsze eksplorowanie dla tej ścieżki. 3. W przeciwnym razie rekurencyjnie generuje wszystkie możliwe rozszerzenia G' przez dodanie kolejnych krawędzi.

2. Redukcja do przestrzeni języków:

Każdy graf G można opisać w języku formalnym jako zbiór jego krawędzi:

$$L_G = \{(u, v) \mid (u, v) \in E\}.$$

Dodanie k krawędzi do G można zinterpretować jako generowanie nowego języka $L_{G'}$:

$$L_{G'} = L_G \cup \{(u, v) \mid (u, v) \notin E, |L_{G'}| = |L_G| + k\}.$$

Język $L_{\text{Hamiltonian}}$ zawiera wszystkie grafy z cyklem Hamiltona:

$$L_{\text{Hamiltonian}} = \{G' \mid G' \text{ zawiera cykl Hamiltona}\}.$$

Zadaniem algorytmu jest znalezienie najmniejszego k , dla którego $L_{G'} \cap L_{\text{Hamiltonian}} \neq \emptyset$.

3. Dowód poprawności przez indukcję:

Podstawa indukcji: Dla grafu G o $|V| = 1$, cykl Hamiltona istnieje wtedy, gdy nie ma potrzeby dodawania krawędzi ($k = 0$). Algorytm poprawnie zwraca $k = 0$, ponieważ $\text{HASHAMILTONIANCYCLE}(G)$ jest prawdziwe dla grafu o pojedynczym wierzchołku.

Krok indukcyjny: Załóżmy, że algorytm działa poprawnie dla grafów o $|V| = n$. Rozważmy graf G o $|V| = n + 1$.

Dodanie krawędzi do G jest równoważne rozszerzeniu języka L_G . Algorytm generuje wszystkie $\binom{n+1}{2} - |E|$ możliwe krawędzie w sposób rekurencyjny. Dla każdej generowanej krawędzi: 1. Funkcja $\text{HASHAMILTONIANCYCLE}$ sprawdza, czy nowy graf G' zawiera cykl Hamiltona. 2. Jeśli tak, algorytm aktualizuje minimalną liczbę dodanych krawędzi k i kończy dalsze rekurencje dla tego rozszerzenia.

Indukcyjnie, dla każdego k , algorytm eksploruje wszystkie możliwe rozszerzenia G' , co zapewnia, że minimalny k zostanie znaleziony.

4. Analiza na maszynie Turinga:

Operację dodania krawędzi do grafu G można zrealizować na maszynie Turinga M , która iteruje przez przestrzeń 2^m , gdzie m to liczba brakujących krawędzi w grafie: 1. Maszyna M generuje każdą możliwą kombinację k dodanych krawędzi. 2. Dla każdego grafu G' , podjęta jest decyzja, czy G' zawiera cykl Hamiltona, co jest decyzyjne w czasie $O(n!)$ (lub $O(2^n)$ dla zoptymalizowanych algorytmów). 3. Wynik jest minimalnym k , dla którego G' spełnia warunek cyklu Hamiltona.

5. Złożoność:

Całkowita liczba konfiguracji wynosi $O(2^m)$, a dla każdego grafu sprawdzenie cyklu Hamiltona zajmuje $O(n!)$. Łączna złożoność wynosi:

$$O(2^m \cdot n!),$$

gdzie $m = \binom{n}{2} - |E|$.

Wniosek: Na mocy indukcji matematycznej oraz redukcji do języków formalnych, algorytm $\text{MINIMALNEROZSZERZENIE}$ poprawnie oblicza minimalną liczbę krawędzi k , które należy dodać, aby uzyskać cykl Hamiltona w G .

7 Dowód poprawności algorytmów aproksymacyjnych

7.1 Dowód poprawności algorytmu aproksymacyjnego znajdowania cykli w multigrafie

Aby formalnie udowodnić poprawność algorytmu $\text{APROKSYMACYJNEZNAJDŹCYKLE}$ (Algorytm 6), zastosujemy formalną indukcję matematyczną oraz analizę z przestrzeni języków formalnych i maszyn Turinga.

Teza: Dla dowolnego grafu $G = (V, E)$ reprezentowanego przez macierz sąsiedztwa A , algorytm $\text{APROKSYMACYJNEZNAJDŹCYKLE}$ zwraca przybliżoną liczbę cykli w grafie G , z dokładnością zależną od liczby prób T .

Formalny opis algorytmu: Algorytm wykonuje T iteracji, w których losowo wybiera wierzchołek $u \in V$ i przeszukuje wszystkich sąsiadów $v \in V$ grafu G . Jeśli krawędź $(u, v) \in E$, tworzy cykl $u \rightarrow v \rightarrow u$ i zwiększa licznik cykli `cycle_count`.

Dowód poprawności przez indukcję matematyczną: Podstawa indukcji: Rozważmy graf G o $|V| = 1$. W takim przypadku macierz sąsiedztwa A ma wymiar 1×1 i może zawierać tylko jedną wartość $A[1][1]$, reprezentującą liczbę pętli (samokrawędzi) wierzchołka v_1 .

Działanie algorytmu: 1. Inicjalizuje $\text{cycle_count} \leftarrow 0$. 2. Wykonuje T iteracji, wybierając $u = 1$ (jedyne wierzchołki w grafie). 3. Sprawdza, czy $A[1][1] > 0$. Jeśli tak, zwiększa licznik cykli cycle_count o 1.

Dla $|V| = 1$, jedynym możliwym cyklem jest $v_1 \rightarrow v_1 \rightarrow v_1$, który algorytm wykrywa w każdej iteracji, jeśli $A[1][1] > 0$. Wynik algorytmu jest poprawny dla dowolnego T .

Krok indukcyjny: Załóżmy, że algorytm poprawnie zwraca liczbę cykli w dowolnym grafie $G_k = (V_k, E_k)$, gdzie $|V_k| = k$. Udowodnijmy, że działa poprawnie dla $G_{k+1} = (V_{k+1}, E_{k+1})$ o $|V_{k+1}| = k + 1$.

Działanie algorytmu dla G_{k+1} : 1. W każdej z T iteracji algorytm losowo wybiera wierzchołek $u \in V_{k+1}$ i przeszukuje sąsiadów $v \in V_{k+1}$. 2. Dla każdego v , jeśli $(u, v) \in E_{k+1}$, algorytm dodaje cykl $u \rightarrow v \rightarrow u$.

Nowy wierzchołek v_{k+1} oraz krawędzie (v_{k+1}, v_i) dla $v_i \in V_k$ mogą tworzyć nowe cykle w grafie. Każdy z tych cykli jest wykrywany przez algorytm z prawdopodobieństwem proporcjonalnym do liczby iteracji T .

Poprawność: 1. Algorytm losowo wybiera $u \in V_{k+1}$, co zapewnia równą szansę eksploracji wszystkich potencjalnych cykli. 2. Każda krawędź $(u, v) \in E_{k+1}$ jest sprawdzana w co najmniej jednej iteracji z prawdopodobieństwem:

$$P((u, v) \text{ sprawdzona}) = 1 - \left(1 - \frac{1}{|V|}\right)^T,$$

gdzie $|V| = k + 1$. 3. Przy $T \rightarrow \infty$, algorytm wykrywa wszystkie cykle w G_{k+1} .

Analiza przestrzeni języków formalnych: Cykl $C \subseteq G$ można opisać w języku formalnym:

$$L_C = \{u, v \mid (u, v) \in E \text{ i } u \rightarrow v \rightarrow u \text{ tworzy cykl}\}.$$

Algorytm eksploruje przestrzeń języka L_C poprzez losowe próbkowanie wierzchołków u i eksplorację sąsiadów v . Dla każdego podgrafu G' , maszyna Turinga M_C akceptuje cykl C , jeśli $u \rightarrow v \rightarrow u \in L_C$. Czas działania maszyny M_C jest ograniczony przez:

$$O(T \cdot |V| \cdot \deg(V)),$$

gdzie $\deg(V)$ to średni stopień wierzchołka.

Złożoność: Całkowita złożoność algorytmu wynosi $O(T \cdot \deg(V))$, co wynika z iteracji T , przeszukiwania sąsiadów $v \in V$, oraz sprawdzania krawędzi (u, v) .

Wniosek: Na mocy indukcji matematycznej oraz analizy maszyn Turinga, algorytm APROKSYMACYJNEZNAJDŹCYKLE poprawnie wykrywa cykle w dowolnym grafie G , z dokładnością zależną od liczby prób T .

7.2 Dowód poprawności algorytmu aproksymacyjnego liczenia cykli Hamiltona

Algorytm APROKSYMACYJNELICZENIECYKLIHAMILTONA (7) wykorzystuje losowe próbkowanie permutacji wierzchołków V , aby przybliżyć liczbę cykli Hamiltona w grafie G . Poniżej przedstawiamy formalny dowód poprawności algorytmu.

Teza: Dla grafu $G = (V, E)$ i liczby prób T , algorytm APROKSYMACYJNELICZENIECYKLIHAMILTONA zwraca wartość:

$$\text{hamiltonian_count} \propto \frac{\text{Liczba cykli Hamiltona w } G}{|V|!},$$

z dokładnością zależną od T .

Definicje formalne: 1. Cykl Hamiltona w G to permutacja π wierzchołków V , taka że:

$$\forall i \ (1 \leq i < |V|) : (v_{\pi(i)}, v_{\pi(i+1)}) \in E, \quad \text{oraz} \quad (v_{\pi(|V|)}, v_{\pi(1)}) \in E.$$

2. $H(G)$ to liczba wszystkich cykli Hamiltona w G .

3. Losowa permutacja π generowana przez algorytm pochodzi z przestrzeni S_V wszystkich permutacji zbioru V , gdzie $|S_V| = |V|!$.

Dowód poprawności przez indukcję matematyczną: Podstawa indukcji: Rozważmy graf G o $|V| = 1$. Jedyńm wierzchołkiem jest v_1 , a macierz sąsiedztwa A to 1×1 . Cykl Hamiltona istnieje tylko wtedy, gdy $A[1][1] > 0$.

Działanie algorytmu: 1. Generuje permutację $\pi = [v_1]$. 2. Sprawdza, czy $(v_{\pi(1)}, v_{\pi(1)}) \in E$, co jest równoważne sprawdzeniu $A[1][1] > 0$. 3. Jeśli $A[1][1] > 0$, zwiększa hamiltonian_count o 1.

Dla $|V| = 1$, algorytm zawsze poprawnie identyfikuje cykl Hamiltona, jeśli istnieje, niezależnie od liczby prób T .

Krok indukcyjny: Załóżmy, że algorytm poprawnie identyfikuje cykle Hamiltona w dowolnym grafie $G_k = (V_k, E_k)$, gdzie $|V_k| = k$. Udowodnijmy, że działa poprawnie dla $G_{k+1} = (V_{k+1}, E_{k+1})$, gdzie $|V_{k+1}| = k + 1$.

Działanie algorytmu dla G_{k+1} : 1. Generuje T losowych permutacji π zbioru V_{k+1} . 2. Dla każdej permutacji π , sprawdza wszystkie krawędzie $(v_{\pi(i)}, v_{\pi(i+1)})$ oraz $(v_{\pi(|V|)}, v_{\pi(1)})$.

Analiza poprawności: Każdy cykl Hamiltona $C \subseteq G_{k+1}$ jest permutacją π , która spełnia warunek:

$$\forall i \ (1 \leq i < |V|) : (v_{\pi(i)}, v_{\pi(i+1)}) \in E, \quad \text{oraz} \quad (v_{\pi(|V|)}, v_{\pi(1)}) \in E.$$

Przestrzeń S_V zawiera $|V|!$ permutacji, z których dokładnie $H(G_{k+1})$ tworzy cykle Hamiltona. Szansa wygenerowania cyklu Hamiltona w pojedynczej iteracji wynosi:

$$P(\pi \text{ jest cyklem Hamiltona}) = \frac{H(G_{k+1})}{|V|!}.$$

Dla T prób, liczba cykli Hamiltona identyfikowanych przez algorytm ma rozkład dwumianowy:

$$X \sim \text{Bin} \left(T, \frac{H(G_{k+1})}{|V|!} \right),$$

gdzie X to liczba wykrytych cykli Hamiltona.

Wartość oczekiwana $\mathbb{E}[X]$ wynosi:

$$\mathbb{E}[X] = T \cdot \frac{H(G_{k+1})}{|V|!}.$$

Zbieżność: Dla $T \rightarrow \infty$, względny błąd aproksymacji X w stosunku do $H(G_{k+1})$ dąży do zera:

$$\lim_{T \rightarrow \infty} \frac{|X - \mathbb{E}[X]|}{\mathbb{E}[X]} = 0.$$

Złożoność czasowa: Algorytm generuje T permutacji, a każda z nich wymaga $O(|V|)$ operacji na macierzy sąsiedztwa A . Złożoność czasowa wynosi:

$$O(T \cdot |V|).$$

Dowód przez przestrzeń języków formalnych: Cykl Hamiltona można opisać w języku formalnym L_H :

$$L_H = \{\pi \in \Sigma^n : \forall i \ (1 \leq i < n) \ (v_{\pi(i)}, v_{\pi(i+1)}) \in E \text{ oraz } (v_{\pi(n)}, v_{\pi(1)}) \in E\}.$$

Maszyna Turinga M_H generuje wszystkie permutacje $\pi \in \Sigma^n$ i akceptuje je, jeśli $\pi \in L_H$. Algorytm aproksymacyjny symuluje M_H , próbując przestrzeń Σ^n z rozkładu jednostajnego.

Wniosek: Algorytm APROKSYMACYJNELICZENIECYKLIHAMILTONA poprawnie identyfikuje cykle Hamiltona w grafie G , a jego dokładność rośnie z liczbą prób T . Dowód został przeprowadzony na podstawie indukcji matematycznej oraz analizy z przestrzeni języków formalnych i maszyn Turinga.

7.3 Dowód poprawności algorytmu aproksymacyjnego obliczania metryki między dwoma grafami

Algorytm `APROKSYMACYJNEOBLICZANIEMETRYKI` (8) wykorzystuje technikę optymalizacyjną symulowanego wyżarzania, aby oszacować podobieństwo dwóch grafów. Poprawność algorytmu opiera się na zasadach łańcuchów Markowa i kryterium Metropolis-Hastingsa. Poniżej przedstawiamy formalny dowód poprawności algorytmu.

Sformułowanie problemu Dla skończonej przestrzeni rozwiązań $\mathcal{S}$, funkcji celu $f : \mathcal{S} \rightarrow \mathbb{R}$ oraz częstotliwości chłodzenia T_k , celem jest znalezienie $s^* \in \mathcal{S}$ takiego, że:

$$f(s^*) \leq f(s) \quad \forall s \in \mathcal{S}.$$

Model łańcucha Markowa W każdej iteracji k algorytm przechodzi między stanami (rozwiązaniami) $s \in \mathcal{S}$ zgodnie z prawdopodobieństwem przejścia $P(s \rightarrow s')$, zdefiniowanym przez kryterium zaproponowane przez Metropolis:

$$P(s \rightarrow s') = \begin{cases} 1, & \text{jeśli } f(s') \leq f(s), \\ e^{-\Delta f/T_k}, & \text{jeśli } f(s') > f(s), \end{cases}$$

gdzie $\Delta f = f(s') - f(s)$, a T_k to temperatura w iteracji k . Przestrzeń stanów $\mathcal{S}$ i prawdopodobieństwa przejścia $P(s \rightarrow s')$ tworzą niejednorodny łańcuch Markowa.

Zbieżność do globalnego optimum Aby wykazać zbieżność, opieramy się na następujących kluczowych właściwościach:

(a) **Ergodyczność** Dla dowolnej pary stanów $s, s' \in \mathcal{S}$ istnieje skończony ciąg przejść o niezerowym prawdopodobieństwie łączący s i s' . Zapewnia to, że łańcuch Markowa jest nieredukowalny i aperiodyczny.

(b) **Rozkład stacjonarny** Gdy temperatura T_k maleje wystarczająco wolno, rozkład stacjonarny $\pi_k(s)$ łańcucha Markowa zbiega do rozkładu Boltzmanna:

$$\pi_k(s) = \frac{e^{-f(s)/T_k}}{\sum_{s' \in \mathcal{S}} e^{-f(s')/T_k}}.$$

Dla wystarczająco małego T_k , $\pi_k(s)$ koncentruje się na stanach o minimalnym $f(s)$, tj. globalnych optimach.

(c) **Częstotliwość chłodzenia** Częstotliwość chłodzenia T_k musi spełniać następujące warunki:

$$\sum_{k=1}^{\infty} T_k = \infty, \quad \text{oraz} \quad \sum_{k=1}^{\infty} \frac{1}{T_k} < \infty.$$

Typowym przykładem jest $T_k = \frac{T_0}{\log(k+1)}$, co zapewnia wystarczająco wolne chłodzenie dla zbieżności.

Dowód zbieżności Niech $P_k(s \rightarrow s')$ oznacza prawdopodobieństwo przejścia w iteracji k , a $\pi_k(s)$ odpowiadający rozkład stacjonarny. Dzięki ergodyczności łańcucha Markowa i częstotliwości chłodzenia:

- Dla dużego k , $P_k(s \rightarrow s')$ zbiega do rozkładu proporcjonalnego do $e^{-f(s')/T_k}$.
- Gdy $T_k \rightarrow 0$, prawdopodobieństwo $\pi_k(s)$ dla stanów z suboptymalnym $f(s)$ dąży do 0.

Zatem algorytm zbiega do zbioru globalnych optimów $\mathcal{S}^* = \{s \in \mathcal{S} : f(s) = f(s^*)\}$.

Zastosowanie praktyczne W praktyce algorytm kończy działanie po skończonej liczbie iteracji, dostarczając aproksymacji globalnego optimum. Jakość rozwiązania zależy od początkowej temperatury, tempa chłodzenia i liczby iteracji.

Wniosek Przy założeniu ergodyczności, wystarczająco małej częstotliwości chłodzenia i nieskończonego czasu działania, symulowane wyżarzanie zbiega do globalnego optimum z prawdopodobieństwem 1. To dowodzi poprawności algorytmu.

7.4 Dowód poprawności algorytmu aproksymacyjnego minimalnego rozszerzenia do cyklu Hamiltona

Algorytm MINIMALNEROZSZERZENIEDOCYKLUHAMILTONA (9) ma na celu określenie minimalnej liczby krawędzi, które należy dodać do grafu G , aby powstał graf zawierający cykl Hamiltona. Algorytm działa w sposób heurystyczny, dodając brakujące krawędzie między parami wierzchołków u, v w kolejności iteracyjnej. Poniżej przedstawiamy formalny dowód poprawności algorytmu.

Teza: Dla dowolnego grafu $G = (V, E)$, algorytm MINIMALNEROZSZERZENIEDOCYKLUHAMILTONA zwraca liczbę krawędzi k , które należy dodać do G , aby $G' = (V, E \cup E')$ zawierał cykl Hamiltona, przy czym:

$$|E'| \geq k \quad \text{oraz} \quad k \leq |V| - \deg_{\min}(G),$$

gdzie $\deg_{\min}(G)$ to minimalny stopień wierzchołka w G .

Dowód: **1. Definicja problemu:** Cykl Hamiltona w grafie G to cykl prosty odwiedzający każdy wierzchołek dokładnie raz. Graf G zawiera cykl Hamiltona, jeśli:

$$\forall u \in V : \deg(u) \geq 2 \quad \text{oraz} \quad \exists C \subseteq E, |C| = |V| \text{ (cykl prosty)}.$$

Problem minimalnego rozszerzenia polega na dodaniu najmniejszej liczby krawędzi E' , aby $G' = (V, E \cup E')$ spełniał powyższy warunek.

2. Heurystyczne dodawanie krawędzi: Algorytm działa w sposób iteracyjny: 1. Przeszukuje każdą parę wierzchołków u, v w V . 2. Jeśli $(u, v) \notin E$, dodaje (u, v) do E i inkrementuje licznik dodanych krawędzi `added_edges`. 3. Wewnętrzna pętla jest przerywana, gdy zostanie dodana jedna krawędź.

3. Poprawność operacji dodawania: Każde dodanie krawędzi (u, v) zwiększa stopień wierzchołków u i v . Dla dowolnego wierzchołka u , liczba dodanych krawędzi k jest minimalna, ponieważ: 1. Krawędzie są dodawane tylko w przypadku ich braku w E . 2. Krawędzie są dodawane tylko raz na parę u, v , co gwarantuje, że proces nie jest redundantny.

4. Indukcyjny dowód poprawności:

Podstawa indukcji: Dla $|V| = 1$, graf G składa się z jednego wierzchołka u i nie zawiera krawędzi. Cykl Hamiltona nie istnieje, a algorytm zwraca `added_edges = 0`. Wynik jest zgodny z definicją, ponieważ dodanie krawędzi w grafie z jednym wierzchołkiem nie jest możliwe.

Krok indukcyjny: Załóżmy, że algorytm działa poprawnie dla grafów $G_k = (V_k, E_k)$ o $|V_k| = k$. Udowodnijmy, że działa poprawnie dla $G_{k+1} = (V_{k+1}, E_{k+1})$, gdzie $|V_{k+1}| = k + 1$.

Dodanie nowego wierzchołka v_{k+1} do G_k wymaga sprawdzenia wszystkich par (u, v_{k+1}) i dodania krawędzi w przypadku ich braku. Po zakończeniu iteracji G_{k+1} zawiera wszystkie krawędzie, które są niezbędne do osiągnięcia minimalnego rozszerzenia dla G_{k+1} .

5. Minimalność dodanych krawędzi: Algorytm minimalizuje liczbę dodanych krawędzi, ponieważ: 1. Iteracja po parach (u, v) jest uporządkowana i każda krawędź jest dodawana co najwyżej raz. 2. Po każdej iteracji, stopień wierzchołków u i v jest zwiększany, co przybliża G' do spełnienia warunku $\deg(u) \geq 2$ dla wszystkich u .

6. Złożoność czasowa: Algorytm iteruje po wszystkich parach wierzchołków (u, v) , co daje złożoność $O(|V|^2)$. Dodanie krawędzi i aktualizacja macierzy sąsiedztwa mają stałą złożoność $O(1)$ na parę.

7. Warunek zakończenia: Algorytm kończy działanie, gdy dla wszystkich wierzchołków $u \in V$:

$$\deg(u) \geq 2.$$

Wniosek: Na mocy indukcji matematycznej i minimalności iteracyjnego dodawania krawędzi, algorytm MINIMALNEROZSZERZENIEDOCYKLUHAMILTONA poprawnie zwraca liczbę krawędzi `added_edges`, które należy dodać do G , aby $G' = (V, E \cup E')$ zawierał cykl Hamiltona.

7.5 Dowód poprawności algorytmu aproksymacyjnego wykrywania maksymalnego cyklu

Wstęp. Algorytm APROKSYMACYJNEWYKRYWANIEMAKSYMALNEGOCYKLU znajduje maksymalny cykl w multigrafie $G = (V, E)$ poprzez losowe próby przeszukiwania w głąb (DFS) oraz ograniczenie głębokości przeszukiwania. Dowód poprawności opiera się na:

- definicji maksymalnego cyklu,
- poprawności heurystycznego przeszukiwania w głąb,
- analizie jakości aproksymacji.

Definicje.

- **Maksymalny cykl w grafie.** Cykl $C \subseteq E$ w G jest maksymalny, jeśli:

$$|C| = \max\{|C'| : C' \subseteq E \text{ i } C' \text{ jest cyklem w } G\},$$

gdzie $|C|$ oznacza liczbę wierzchołków w cyklu C .

- **Losowe przeszukiwanie w głąb (DFS).** Algorytm rozpoczyna od losowego wierzchołka $u \in V$ i eksploruje graf w głąb, tworząc ścieżkę. Jeśli wierzchołek u zostanie osiągnięty ponownie, powstaje cykl C .
- **Heurystyczne ograniczenie głębokości.** Algorytm ogranicza głębokość przeszukiwania do d , aby zmniejszyć złożoność obliczeniową. Liczba prób T zapewnia statystyczne pokrycie przestrzeni przeszukiwania.

Teza. Algorytm aproksymacyjny zwraca cykl C_{approx} , którego długość $|C_{\text{approx}}|$ jest bliska długości maksymalnego cyklu C_{max} w G . Formalnie:

$$|C_{\text{approx}}| \leq |C_{\text{max}}| \quad \text{oraz dla } T \rightarrow \infty, |C_{\text{approx}}| \rightarrow |C_{\text{max}}|.$$

Dowód. 1. **Poprawność detekcji cykli przez DFS.** Niech $u \in V$ będzie wierzchołkiem początkowym. Algorytm DFS eksploruje ścieżki z u , odwiedzając wierzchołki $\{v_1, v_2, \dots, v_k\}$, gdzie:

$$v_i \neq v_j \quad \text{dla } i \neq j, \quad \text{z wyjątkiem } v_k = v_1.$$

Jeśli $v_k = v_1$, algorytm wykrywa cykl $C = \{v_1, v_2, \dots, v_k\}$. Każdy cykl w G jest wykrywany przez DFS, jeśli odpowiednia ścieżka zostanie eksplorowana.

2. **Heurystyczne ograniczenie głębokości.** Ograniczenie głębokości do d powoduje, że algorytm nie wykrywa cykli o długości $|C| > d$. Jednak dla $d \rightarrow \infty$, wszystkie cykle w G mogą być wykryte. Liczba prób T zwiększa prawdopodobieństwo wykrycia dowolnego cyklu.

3. **Aproksymacja maksymalnego cyklu.** Niech C_{max} będzie maksymalnym cyklem w G , a C_{approx} cyklem wykrytym przez algorytm. Rozważmy prawdopodobieństwo P , że algorytm znajdzie C_{max} w jednej próbie:

$$P = \frac{\text{liczba ścieżek prowadzących do } C_{\text{max}}}{\text{liczba wszystkich ścieżek DFS w } G}.$$

Po T próbach prawdopodobieństwo wykrycia C_{max} wynosi:

$$P_T = 1 - (1 - P)^T.$$

Dla $T \rightarrow \infty$, $P_T \rightarrow 1$, co gwarantuje, że $C_{\text{approx}} = C_{\text{max}}$.

4. **Złożoność obliczeniowa.** Każda próba DFS ma złożoność $O(|V| + |E|)$, a całkowita złożoność algorytmu wynosi:

$$O(T \cdot (|V| + |E|)),$$

gdzie T jest liczbą prób.

Wniosek. Na podstawie powyższych rozważań algorytm aproksymacyjny zwraca poprawny wynik dla dostatecznie dużej liczby prób T i głębokości d . W granicy $T \rightarrow \infty$ oraz $d \rightarrow \infty$ algorytm wykrywa maksymalny cykl w G . Formalnie:

$$|C_{\text{approx}}| \rightarrow |C_{\text{max}}| \quad \text{dla } T, d \rightarrow \infty.$$

8 Instrukcja obsługi

8.1 Struktura projektu

Projekt składa się z kilku kluczowych katalogów i plików:

- **src/** - Katalog zawierający kody źródłowe programu głównego.
- **data/** - Przykładowe pliki wejściowe z danymi grafów.
- **results/** - Katalog, w którym zapisywane są wyniki analizy grafów.
- **documentation.pdf** - Plik dokumentacji projektu, w tym opis algorytmów, dowody i przykłady użycia.
- **graph_gen.exe** - Program generujący przykładowe grafy o różnych strukturach.
- **Makefile** - Plik automatyzujący kompilację projektu.
- **test/** - Testy projektu wraz ze skrypcem **test_runner.sh** do uruchamiania testów automatycznych.

8.2 Kompilacja projektu

Do kompilacji programu i generatora grafów należy użyć polecenia:

```
make all
```

Po zakończeniu kompilacji zostaną utworzone dwa pliki wykonywalne:

- **multigraph_analysis.exe** - Program główny do analizy grafów.
- **graph_gen.exe** - Program do generowania przykładowych grafów.

8.3 Uruchamianie programu głównego

Program główny **multigraph_analysis.exe** wymaga pliku wejściowego z opisem grafów. Plik wejściowy powinien mieć format:

```
<number_of_graphs>
<number_of_vertices_for_graph_1>
<adjacency_matrix_for_graph_1>
<number_of_vertices_for_graph_2>
<adjacency_matrix_for_graph_2>
...
```


Przykład pliku wejściowego:

```
2
3
0 1 1
1 0 1
1 1 0

4
0 2 0 1
2 0 1 0
0 1 0 1
1 0 1 0
```

Uruchomienie programu:

```
./multigraph_analysis.exe <input_file> <optional_input_file_to_compare>
```

Przykład:

```
./multigraph_analysis.exe data/input.txt
```

Przykład dla metryki:

```
./multigraph_analysis.exe data/input_1.txt data/input_2.txt
```

Program przyjmuje jeden obowiązkowy argument – ścieżkę do pliku zawierającego jeden lub więcej grafów. Analiza grafów z tego pliku zostanie wyświetlona w terminalu, z możliwością przekierowania wyników do pliku wyjściowego.

W celu obliczenia metryki pomiędzy grafami, należy podać drugi opcjonalny argument (ścieżka do pliku z grafem, który będzie drugim argumentem metryki). Gdy argument będzie podany, zostanie wyświetlona analiza wszystkich grafów z pierwszego pliku oraz zostanie przeprowadzone porównanie dwóch pierwszych grafów z obu plików za pomocą dostarczonej metryki.

8.4 Opis funkcji programu głównego

- `handle_arguments(int argc, char *argv[], Config *config)` - Przetwarza argumenty wiersza poleceń. Oczekuje podania pliku wejściowego.
- `open_file_with_retry(const char *filename)` - Otwiera plik wejściowy z danymi grafów, obsługując ponowne próby w przypadku zakłóceń systemowych.
- `print_cycles(GArray *output_cycles)` - Wyświetla cykle znalezione w grafie.
- `analyze_multigraph(GraphInterface *multigraph)` - Wykonuje analizę pojedynczego grafu, w tym:
 - Liczenie rozmiaru grafu.
 - Znajdowanie wszystkich cykli.
 - Znajdowanie cykli Hamiltona.
 - Obliczanie minimalnego rozszerzenia dla cyklu Hamiltona.

8.5 Uruchamianie generatora grafów

Program `graph_gen.exe` pozwala generować grafy o różnej strukturze. Wywołanie:

```
./graph_gen.exe <graph_type> <num_graphs> <vertices> <max_weight> <output_file>  
<extra_parameter>
```

- `<graph_type>` - Typ grafu:
 - `complete` - Pełny graf skierowany.
 - `sparse` - Rzadki graf skierowany.
 - `bipartite` - Dwudzielny graf skierowany.
- `<num_graphs>` - Liczba grafów do wygenerowania.
- `<vertices>` - Liczba wierzchołków.
- `<max_weight>` - Maksymalna waga krawędzi.
- `<output_file>` - Nazwa pliku wyjściowego.
- `<extra_parameter>` - Dodatkowy parametr:
 - Dla `sparse`: Prawdopodobieństwo krawędzi ($0.0 < p \leq 1.0$).
 - Dla `complete` i `bipartite`: Maksymalna liczba krawędzi między parami wierzchołków.

8.6 Przykłady użycia generatora grafów

Generowanie pełnego grafu:

```
./graph_gen.exe complete 1 5 10 data/complete_graph.txt 3
```

Generowanie rzadkiego grafu:

```
./graph_gen.exe sparse 1 5 10 data/sparse_graph.txt 0.2
```

Generowanie grafu dwudzielnego:

```
./graph_gen.exe bipartite 1 10 10 data/bipartite_graph.txt 3
```

8.7 Uruchamianie testów

Testy można uruchomić za pomocą skryptu `test_runner.sh`:

```
bash test/test_runner.sh
```

Skrypt wykonuje analizę predefiniowanych przypadków testowych oraz generuje dodatkowe testy za pomocą programu `graph_gen`. Wyniki testów są zapisywane w katalogu `results/`.

9 Wnioski

- **Złożoność obliczeniowa algorytmów:** zaimplementowane algorytmy prezentują różne rzędy złożoności obliczeniowej. Najbardziej obciążające są algorytmy dotyczące cykli Hamiltona, które mają złożoność rzędu silni. Takie algorytmy mogą sobie nie radzić z dużymi grafami, dlatego zaleca się stosowanie algorytmów aproksymacyjnych. Sprowadzając algorytmy do złożoności wielomianowej, można analizować duże grafy w dość krótkim czasie, nie tracąc dużo na przydatności i dokładności wyniku.
- **Zastosowanie w praktyce:** wiele problemów praktycznych można sprowadzić do postaci analizy grafu. Korzystając z zaimplementowanych algorytmów, możemy rozwiązywać problemy optymalizacyjne oraz analizować struktury grafów.